OOP – Aufgabentypen und Fehlerjagd

Kapitel 1–4 · Übungsblätter OOP01–OOP04 und Skript · C++17 · Google Test · pbma · Stand 03.11.2026

Inhalt

Teil 1 Musteranalyse: Gemeinsamkeiten der Blätter · 10 Aufgabentypen · Signalwörter → Code · versteckte Prüfpunkte · Lese-Routine (an `Time` durchgespielt) · Was tun, wenn du es nicht aus dem Kopf kannst

Teil 2 Neun Trainingsaufgaben A1–A9: Fehlerjagd, Header aus Test, Orakel, Debugger-Fragen, Stufenbau

Teil 3 Rezeptkarten R1–R9 · **Teil 4** Fehlerbuch-Vorlage

Auflösung separat: `loesungen/OOP_Fehlerjagd_Aufloesung.pdf`

Teil 1 – Musteranalyse: So sind die Aufgaben gestellt

Grundlage: die Übungsblätter OOP01–OOP04 (Winter 26/27), die vorgegebenen Tests (`test_tausche.cpp`, `test_at.cpp`, `test_intvector1.cpp`) und die Beispiele im Skript, Kapitel 2–4. Seitenangaben beziehen sich auf die gedruckten Seitenzahlen im Skript.

1.1 Was alle Blätter gemeinsam haben

1. **Die Aufgaben bauen aufeinander auf.** `Time` → `Participant` mit `Time` → `Participant` mit static-Zähler (OOP03). `IntVector` → Kopie und Zuweisung → derselbe Code im Unit-Test (OOP04). Im Testat heißt das: Ohne funktionierende Basisklasse kommst du bei den Folgeaufgaben nicht weiter. Bring deshalb zuerst die Basis zum Laufen.
2. **Die Spezifikation steht im Fließtext.** OOP03 A1 sagt es wörtlich: *„Die Prototypen der Methoden sind nicht angegeben, da Sie die korrekten Prototypen aus dem Text herauslesen sollen.“* Auch wo Prototypen vorgegeben sind (OOP04: `size_t size()`, `int& at(size_t idx)`), fehlt manchmal das `const`. Ob es nötig ist, zeigt der Test.
3. **Fehlertexte sind exakt vorgegeben.** `"index overflow"` (OOP02 A4), `"index out of range"` (OOP04 A1). Der offizielle Test prüft den Text mit `EXPECT_STREQ(e.what(), "index out of range")`. Ein Tippfehler macht den Test rot.
4. **Es gibt immer dasselbe Werkzeug:** `pbma` (`args_t`, `read_ints`, `create_same_randints`, `Timer`), Google Test mit `gtest_main.cpp` und `alltest.h`, valgrind und den Debugger.
5. **Verifizieren gehört zur Aufgabe.** *„Berechnen Sie … mit der Hand … Debuggen Sie … verifizieren Sie“* (OOP01 A2), *„Verfolgen Sie im Debugger … wie oft und wann der Zähler erhöht … wird“* (OOP03 A3), *„Debuggen Sie die Aufrufe“* (OOP04 A2), *„Lassen Sie … mit valgrind laufen“* (OOP04 A1).
6. **Manche Aufgaben brauchen keinen Rechner:** *„Übersetzt es korrekt? … Gehen Sie davon aus, dass nach jedem Fehler die entsprechende Zeile einfach fehlt“* (OOP02 A2), *„Wie groß … ist ein Person-Objekt?“* (OOP01 A4), *„Schreiben Sie Code so kompakt wie möglich“* (OOP03 A5, A6).
7. **Randfälle stehen versteckt im Text.** OOP04 A2 verlangt *„ein neues Feld v3 der Größe 10 und weisen diesem die Kopie zu“*. Das ist eine Zuweisung zwischen verschieden großen Feldern. Der offizielle Test prüft außerdem `v1 = v2 = v3 = v4` und den Index `-1` als `size_t`.

1.2 Die zehn Aufgabentypen

Typ	Auf den Blättern	Im Skript	Erkennst du an	Training
T1 Kommandozeilen-Programm mit Datei	OOP01 A3, OOP02 A1, A5, OOP04 A1 (`-crash`)	2.9 Ein-/Ausgabe S. 15–19, *Zeilen nummerieren* S. 19	„auf der Kommandozeile“, „Flag -v“, „Vorgabe ist die Datei“, „zwei Nachkommastellen“	A2
T2 Freie Funktion mit Referenz/const, Test gegeben	OOP02 A3 `tausche`, A4 `at`	2.1 const S. 3, 2.2 Referenzen S. 4–6	„liefert eine Referenz“, „Referenzparameter“, „Testen Sie mit test_….cpp“	A1
T3 Compiler-Orakel	OOP02 A2	2.3 Überladen S. 6, 2.4 Default-Werte S. 7	„Übersetzt es korrekt?“	A5 a
T4 Klasse aus Fließtext	OOP03 A1 `Time`	3.1 `Date` S. 23 ff., 3.2 Konstruktoren S. 29 ff.	„Prototypen nicht angegeben“, „const wann immer sinnvoll“	A3, A7
T5 Klasse benutzt Klasse, vector, pbma	OOP03 A2 `Participant`	3.3 Objekte S. 34 ff.	„vector<…> aus den Namen der Kommandozeile“	A9
T6 static und Lebenszyklus	OOP03 A3	3.4 static S. 39, `Noisy` S. 35	„statisches Attribut“, „aktuell gültigen“, „verfolgen“	A4
T7 Kompakt / rückwärts	OOP03 A4–A6	`Integer/explicit` S. 31, Initialisierungsliste/Delegation S. 33	„so kompakt wie möglich“, Ausgabe steht als Kommentar	A5 b, c
T8 Freispeicher und Dreierregel	OOP04 A1–A3 `IntVector`	2.8 Freispeicher S. 13, `String` S. 43–50	„dynamisch anzulegende“, „eigenen Freispeicher“, „kein Speicherleck“	A6
T9 Ressource, nicht kopierbar	OOP04 A4 `Logger`	2.9 Streams S. 15 ff., 4.5 Dreierregel S. 50	„kann man nicht kopieren oder zuweisen“, „Strom wird geschlossen“	A7
T10 Speicher und Zeiger rechnen	OOP01 A4	2.8, Abb. 2.1 S. 15	„Wie groß …?“, „Was ist die Ausgabe …“ mit Zeigerdifferenzen	A8

1.3 Signalwörter: vom Satz zum Code

Formulierung im Text	Bedeutet im Code	Quelle
„Wenn ein Attribut nicht angegeben wird, dann ist es 0“	Default-Parameter `Time(int h = 0, int m = 0, int s = 0)`	OOP03 A1
„so wenige Methoden wie möglich“	Default-Parameter statt mehrerer Konstruktoren	OOP03 A4
„auch für konstante …“, „const wann immer sinnvoll“, `const A ca1;` im Beispiel	`const` hinter lesenden Methoden	OOP03 A1, A4
„liefert eine Referenz“, „so, dass der Aufrufer es verändern kann“, `a.get_val() = 42;`	Rückgabe `T&` (plus `T … const` für const-Objekte)	OOP02 A4, OOP03 A4

Formulierung im Text	Bedeutet im Code	Quelle
„ohne Kopie zurückgegeben“	Rückgabe `const T&`	Probetestat-Stil
„bricht … mit der Fehlermeldung X ab“	`throw std::runtime_error("X");` mit exaktem Text	OOP02 A4, OOP04 A1
„falsch zurückgegeben“, „liefern zurück, ob … möglich war“	Rückgabetyp `bool`	OOP03 A1
„Bei Misserfolg ändert sich nichts“	erst prüfen, dann ändern	Probetestat-Stil
„werden … normiert“	Wert auf die Grenze setzen (clamp), nicht werfen	OOP04 A4
„statisches … Attribut“, „unabhängig von einer Instanz“	`static` Attribut + Definition in der `.cpp`, `static` Methode	OOP03 A3
„greift direkt auf die privaten Attribute zu“	`friend`	Skript 3.5
„kann man nicht kopieren oder zuweisen“	`X(const X&) = delete; X& operator=(const X&) = delete;`	OOP04 A4
„soll nicht unbemerkt umgewandelt werden“	`explicit`	Skript S. 31
„unveränderlich“, „kann nach der Konstruktion nicht verändert werden“	`const`-Attribut, nur über Initialisierungsliste	Skript S. 33
„Elementinitialisierungsliste so oft wie möglich“	alles in `: a(…), b(…)`, Rumpf leer	OOP03 A4
„dynamisch anzulegende(s) … Array“	`int* _daten = new int[n]`, Destruktor mit `delete[]`	OOP04 A1
„in der neuen Instanz … ihren eigenen Freispeicher“	tiefe Kopie im Kopierkonstruktor	OOP04 A2
„kein Speicherleck“ beim Zuweisen	alten Speicher freigeben (oder Copy & Swap)	OOP04 A2
„Erstes Argument … wenn nicht angegeben, dann X“	`args.pos(0, "X")`	OOP02 A5
„Falls das Flag -v gegeben ist“	`args.flag("v")` (ohne Minus)	OOP02 A1, A5
„nur wenn in der Kommandozeile das Flag -crash gesetzt ist“	`if (args.flag("crash")) crash();`	OOP04 A1
„durchnummeriert“, „zeilenweise“	Schleife mit Index, `setw`	OOP02 A5, Skript S. 19
„auf zwei Nachkommastellen genau“	`fixed << setprecision(2)`, vorher nach `double` wandeln	OOP02 A5
„Reservieren Sie … ausreichend, auch nicht zu viel“	erst Anzahl lesen, dann `new int[anzahl]`	OOP02 A5
„Vergleichen Sie die Ausführungszeiten“	`Timer timer; … timer.human_measure()`	OOP01 A3
„gleiche Zufallszahlen“, „zum Testen“	`create_same_randints(n, lo, hi)`	OOP01 A4

1.4 Was Tests prüfen, ohne dass es im Text steht

- **Aufrufe auf const-Objekten.** Ein `const X x;` oder ein Parameter `const X&` im Test braucht `const`-Methoden. Sonst kommt der Fehler `discards qualifiers`.
- **Referenz statt Kopie.** Prüfbar durch Zuweisung (`at(v, 10, 5) = 999`) oder durch den Vergleich von Adressen (`EXPECT_EQ(&r1, &r2)`).

- **Indexgrenzen.** `size`, `size + 1000000` und `-1` als `size_t`, also eine riesige Zahl (`test_intvector1.cpp`: `check_fail(v, -1)`).
- **Zuweisung.** Verschiedene Größen, Selbstzuweisung und Kettenzuweisung `v1 = v2 = v3 = v4`.
- **Exception im Konstruktor.** Das Objekt existiert danach nicht, sein Destruktor läuft nicht. Zähler dürfen also erst nach allen Prüfungen hochzählen.
- **Speicher.** `TEST_F(AllTest, …)` zählt jedes `new`/`delete` und jedes `new[]`/`delete[]`. Ein falsches Paar fällt sofort auf (`freed_not_allocated`, `allocated_not_freed`).
- **Ausgabe zeichengenau.** Leerzeichen, `setw`-Breite und Nachkommastellen müssen stimmen.

> **Kurs-Konvention bei AllTest:** In den offiziellen Tests steht jedes `EXPECT` in einem Block `{ SuspendMemCheck smc; … }`. Halte dich in eigenen Tests mit `TEST_F(AllTest, …)` daran. In unserer Testumgebung (gtest 1.14) beendet ein fehlschlagendes `EXPECT` ohne diesen Block das ganze Testprogramm mit *Segmentation fault*, direkt nach der ersten Failure-Meldung. Für schnelle eigene Versuche nimmst du deshalb einfach `TEST(Mein, …)` ohne Speicherprüfung.

1.5 Lese-Routine für einen Aufgabentext (3 Minuten)

1. **Nomen anstreichen:** Daraus werden Klassen und Attribute, mit Typ (Text → `std::string`, Anzahl → `size_t`, Geld → `long` in Cent).
2. **Verben anstreichen:** Daraus werden Methoden. Lesend oder ändernd? Lesend bedeutet `const`.
3. **„kann/darf nicht“, „immer“, „zwischen … und …“:** Das sind Invarianten. Sie werden zu Prüfungen im Konstruktor und in Settern, mit exaktem Fehlertext. Bei `= delete` und `explicit` steht es ebenfalls so im Text.
4. **Beispiele und Ausgaben:** Das sind deine ersten Testfälle. Schreib sie dir als `EXPECT_EQ` auf.
5. **Header schreiben, bevor du Logik schreibst.** Danach Stubs, sodass alles kompiliert.

Durchgespielt an OOP03 A1 (`Time`):

Satz im Text	Deklaration
„Zeit bestehend aus Stunden, Minuten und Sekunden“	`int _h, _m, _s;`
„… in dieser Reihenfolge angeben. Wenn ein Attribut nicht angegeben wird, dann ist es 0.“	`Time(int h = 0, int m = 0, int s = 0);`
„Minuten und Sekunden sind immer zwischen 0 und 59“	Invariante: im Konstruktor normalisieren (Überlauf in die nächste Einheit)
„mit den Methoden hour, minute und second einzeln zugreifen“	`int hour() const;` `int minute() const;` `int second() const;`
„einer Zeit eine andere Zeit mit add hinzufügen“	`bool add(const Time& other);`
„add_hours … Parameter auch negativ … auf den Anfang gesetzt und falsch zurückgegeben“	`bool add_hours(int h);` (ebenso `add_minutes`, `add_seconds`)
„nur die Sekunden … seit Anfang der Zeit … justseconds“	`long justseconds() const;`
„diff … positiv, wenn der Parameter vor der Zeit liegt“	`long diff(const Time& other) const;` → `justseconds() - other.justseconds()`
„für Menschen einfach lesbar ausgeben“	`void out(std::ostream& os = std::cout) const;`

1.6 Wenn du es nicht aus dem Kopf programmieren kannst

1. Den Test als Spezifikation lesen. Jede Testzeile verrät eine Deklaration:

Steht im Test	Folgerung
`const X x; x.f();` oder `void g(const X& x) { x.f(); }`	`f` ist `const`
`x.f() = 5;`	`f` liefert `T&`. Wird `f` auch auf const-Objekten aufgerufen, brauchst du zusätzlich `T f() const`
`EXPECT_EQ(&x.name(), &x.name())`	Rückgabe `const std::string&`
`X::f()` ohne Objekt	`static`
`EXPECT_THROW(X(0), std::runtime_error)`	Konstruktor prüft und wirft
`X x; X y(3); X z(3, 4);`	ein Konstruktor mit Default-Parametern oder Delegation
`static_assert(!is_convertible<int, X>::value)`	Konstruktor `explicit`
`static_assert(!is_copy_constructible<X>::value)`	`X(const X&) = delete;`
freie Funktion `f(a, b)` ändert private Attribute	`friend` in der Klasse deklarieren
`EXPECT_EQ(t.size(), 3u)`	Rückgabetyp `size_t` (vorzeichenlos, deshalb `3u`)
`EXPECT_DOUBLE_EQ(t.wert(), 2.5)`	Rückgabetyp `double`

2. Den Compiler fragen. Schreib den Aufruf so, wie er im Test steht, und bau. `no matching function for call to …` listet darunter die Kandidaten auf, `discards qualifiers` meldet fehlendes `const`. Die Meldung sagt dir, was fehlt.

3. Mit einem Gerüst starten, nicht mit einer leeren Datei. Die Rezeptkarten in Teil 3 sind zum Abschreiben und Anpassen gedacht. Kopierkonstruktor, Copy & Swap, ein static-Zähler oder ein pbma-`main` sind jedes Mal fast gleich.

4. Im Skript nachschlagen, falls Unterlagen erlaubt sind (vorher klären):

Gesucht	Skript
Referenzparameter, const-Referenz	2.2, S. 4–6
Überladung, Default-Werte	2.3–2.4, S. 6–8
`new[]`/`delete[]`, 2D-Feld	2.8, S. 13–15 (Abb. 2.1)
Datei lesen, `getline`, `setw`	2.9, S. 15–19
Klasse mit Header/Implementierung (`Date`)	3.1, S. 23–28
Konvertierungskonstruktor, `explicit`	3.2, S. 31–32
Initialisierungsliste, Delegation	3.2, S. 33–34
Lebenszyklus (`Noisy`)	3.2, S. 35
`static`	3.4, S. 39
`friend`	3.5, S. 40
Zuweisung, Kopierkonstruktor (`String`)	4.2–4.3, S. 44–48
Copy & Swap, Dreierregel	4.4–4.5, S. 49–50

5. Punkte sichern, in dieser Reihenfolge: Das Projekt kompiliert mit Stubs. → Einfache Tests werden grün. → Randfälle und Exceptions. → Speicher (AllTest, valgrind). Ein kompilierendes Projekt mit 6 von 10 grünen Tests ist mehr wert als eine fast fertige Lösung, die nicht baut.

6. Zeitgrenze. Kommst du nach 15 Minuten an einem Fehler nicht weiter, schreib dir die Testnummer und deine Vermutung auf und mach mit der nächsten Aufgabe weiter. Mit frischem

Kopf findest du den Fehler danach oft schneller.

Teil 2 – Trainingsaufgaben

Ordner: `Testat_1/Fehlerjagd_Training/aufgaben/`, Hilfsdateien in `vorlagen/`, Lösungen und Auflösung in `loesungen/`. Diesen Ordner öffnest du erst, wenn du fertig bist.

So arbeitest du mit diesem Teil

Modus	Was du tust	Was du dabei trainierst
Fehlerjagd	Code und Tests sind gegeben, die Zahl der Fehler steht dabei. Finde sie **nur** mit Testausgabe, Debugger und valgrind.	Fehler lokalisieren statt raten
Header aus Test	Nur der Test ist gegeben. Leite Header und Implementierung daraus ab.	Prototypen erkennen, wenn du sie nicht im Kopf hast
Orakel	Erst auf Papier vorhersagen, dann prüfen.	Compilerregeln und Lebenszyklus sicher beherrschen
Debugger-Frage	Beantworte die Frage, indem du im Debugger nachsiehst.	Breakpoints, Expressions, Call-Stack
Stufenbau	Eine Stufe nach der anderen bauen, jede mit eigener Testdatei.	Kleine Schritte, immer ein grüner Stand

Regeln für die Fehlerjagd

- Erst das Symptom aufschreiben (welcher Test, was erwartet, was kam heraus), dann suchen.
- Jeder Fund kommt als eine Zeile ins Fehlerbuch (Vorlage in Teil 4): Symptom, Ursache, Werkzeug.
- Behebe immer nur einen Fehler, dann baust und testest du neu.
- Die Fehler sind typische Testat-Fehler, kein absichtlich unleserlicher Code. Nicht jede Funktion ist fehlerhaft.

Projekt-Setup wie auf Blatt 2: pro Aufgabe ein Projekt, `alltest.h` und `gtest_main.cpp` (bzw. `pbma.h/.cpp`) aus `vorlagen/` hineinkopieren, Linker `-pthread` und Library `gtest`. Programme mit eigener `main` kommen in ein eigenes Projekt ohne Tests.

Empfohlene Reihenfolge

Nr	Aufgabe	Typ	Modus	Zeit
A1	Feldfunktionen	T2	Fehlerjagd (3 Fehler)	15 min
A5	Compiler-Orakel und Kompakt	T3, T7	Orakel	20 min
A3	Datum	T4	Fehlerjagd (4 Fehler)	25 min
A4	Ticket und Lebenslauf	T6	Fehlerjagd (2 Fehler) + Orakel	25 min
A8	Speicherlayout	T10	Debugger-Frage	15 min
A2	notenstat	T1	Fehlerjagd (3 Fehler)	15 min
A6	Zahlenfolge	T8	Fehlerjagd (4 Fehler)	30 min
A7	Thermostat	T4, T9	Header aus Test	30 min
A9	Staffel	T5	Stufenbau	40 min

A1 – Feldfunktionen (T2 · Fehlerjagd · 15 min)

Dateien: `a1_feld/feld.h`, `feld.cpp`, `test_feld.cpp`

Eine kleine Bibliothek arbeitet auf `std::vector<int>` (Spezifikation in `feld.h`):

- `grenzen(v, mn, mx)` ermittelt Minimum und Maximum. Bei leerem Vektor wird `"empty"` geworfen, `mn` und `mx` bleiben dann unverändert.
- `groesstes(v)` liefert das größte Element so, dass der Aufrufer es im Vektor ändern kann. Bei Gleichstand gilt das erste.
- `entferne(v, wert)` entfernt alle Vorkommen und liefert deren Anzahl.
- `tausche(a, b)` vertauscht zwei `int`-Zeiger (wie Blatt 2, Aufgabe 3).

Auftrag: `feld.cpp` enthält **3 Fehler**. Finde und behebe sie, bis alle 10 Tests grün sind.

Hinweise: - Lies zuerst die **Warnungen** beim Übersetzen, nicht nur die Fehler. - Das Testprogramm bricht an einer Stelle einfach ab. Starte es dann mit *Debug As*: Der Debugger hält an der Absturzstelle, und im Call-Stack siehst du, welcher Test gerade lief. - Für `entferne`: Spiel `{2, 2, 1, 2, 2}` auf Papier durch und notiere nach jedem `erase` den Index `i`, die Länge und den Inhalt.

Transfer: Warum darf `grenzen` den Vektor als `const std::vector<int>&` bekommen, `groesstes` aber nicht?

A2 – notenstat (T1 · Fehlerjagd · 15 min)

Dateien: `a2_notenstat/notenstat.cpp`, `punkte.dat`, `wenig.dat`, `kaputt.dat`. Eigenes Projekt mit `pbma.h/.cpp`.

So soll das Programm arbeiten:

- Erstes Positionsargument ist der Dateiname, Vorgabe `punkte.dat`. Die erste Zahl in der Datei ist die Anzahl der folgenden Punktzahlen. Passt sie nicht, bricht das Programm mit einer Meldung ab.
- `-grenze=N` setzt die Bestehensgrenze (Vorgabe 50, erlaubt 0 bis 100).
- Mit dem Flag `-v` werden zuerst alle Punktzahlen durchnummeriert ab 1 ausgegeben.
- Danach folgen Teilnehmerzahl, Anzahl Bestandene, Durchschnitt mit zwei Nachkommastellen und die beste Punktzahl mit ihrer Nummer (1-basiert, bei Gleichstand die erste).

Erwartete Ausgaben:

```
$ ./notenstat -v                    $ ./notenstat -grenze=70
Nr  1:  78                          Teilnehmer: 10
Nr  2:  45                          Bestanden (>= 70): 5
...                                 Durchschnitt: 67.20
Nr 10:  70                          Beste Punktzahl: 97 (Nr 6)
Teilnehmer: 10
Bestanden (>= 50): 7                $ ./notenstat wenig.dat
Durchschnitt: 67.20                 Teilnehmer: 3
Beste Punktzahl: 97 (Nr 6)          Bestanden (>= 50): 2
                                    Durchschnitt: 49.67
                                    Beste Punktzahl: 50 (Nr 1)
```

Auftrag: Das Programm übersetzt ohne Warnung, enthält aber **3 Fehler**. Finde sie, indem du die Ausgabe mit der Erwartung vergleichst und im Debugger die beteiligten Variablen ansiehst. Kommandozeilenargumente setzt du in Eclipse unter *Run Configurations → Arguments*.

Transfer: Wie würdest du das Programm umbauen, damit `auswerten` ohne Konsole testbar ist? (Stichwort: Ergebnis zurückgeben statt ausgeben, oder `std::ostream&` als Parameter.)

A3 – Datum (T4 · Fehlerjagd · 25 min)

Dateien: `a3_datum/Datum.h`, `Datum.cpp`, `test_datum.cpp`

Wie die Klasse `Date` im Skript (3.1), mit gregorianischem Kalender:

- Konstruktor mit Tag, Monat, Jahr. Die Vorgabe ist der 01.01.1970, und man kann von hinten weglassen. Ungültige Daten werfen `"invalid date"`.
- `tag()`, `monat()`, `jahr()` sind auch für konstante Daten aufrufbar.
- `schaltjahr(jahr)` und `tage_im_monat(monat, jahr)` sind ohne Instanz aufrufbar. Schaltjahr: durch 4 teilbar, aber nicht durch 100, es sei denn durch 400.
- `naechster_tag()` und `addiere_tage(n)` (negatives `n` wirft `"negative days"`), `vor(other)` (liegt dieses Datum echt vor `other`?) und `out(os)` im Format `TT.MM.JJJJ`.

Auftrag: Es gibt **4 Fehler**. Einer davon verhindert schon das Übersetzen. Die anderen drei findest du über die roten Tests.

Hinweise: - Bei der Compiler-Meldung zählt die erste `error:`-Zeile. Darunter zeigt `note: in call to …` auf die Deklaration. - Für `vor`: Stell eine Tabelle mit den Testfällen auf, deinem erwarteten Ergebnis und dem Ergebnis des Codes. Welcher Fall weicht als erster ab? - Für den Jahreswechsel: Setz einen bedingten Breakpoint in `naechster_tag` mit der Condition `_monat == 12`.

Transfer: Warum sind `schaltjahr` und `tage_im_monat` `static`? Was wäre der Nachteil, wenn sie normale Methoden wären?

A4 – Ticket und Lebenslauf (T6 · Fehlerjagd + Orakel · 25 min)

Teil a – Ticket (Fehlerjagd, 2 Fehler)

Dateien: `a4_ticket/Ticket.h`, `Ticket.cpp`, `test_ticket.cpp`

Tickets bekommen fortlaufende Nummern ab 1. `Ticket::aktive()` zählt alle aktuell existierenden Tickets. Eine Kopie gilt als weiteres aktives Ticket mit derselben Nummer. Ein Ticket ohne Kunde wirft `"kein Kunde"`, und ein fehlgeschlagenes Ticket verbraucht keine Nummer.

Auftrag: Die Tests `Fehlschlag`, `Kopie` und `Wertparameter` sind rot. Dahinter stecken **2 Fehler**. Verfolge `_aktive` im Debugger: Setz Breakpoints in Konstruktor und Destruktor und zähle mit, wie in Blatt 3, Aufgabe 3. Ein Breakpoint im Kopierkonstruktor geht nicht, weil es keinen gibt. Was folgt daraus?

Teil b – Lebenslauf (Orakel, ohne Fehler)

Datei: `a4_ticket/lebenslauf.cpp`, eigenes Konsolenprojekt. Die Klasse `Laut` meldet sich bei Konstruktion `K(…)`, Kopie `C(…)`, Zuweisung `Z(…)` und Destruktion `D(…)`. Eine Kopie bekommt einen Strich an den Namen (`a'`).

1. Schreib für die Zeilen 1 bis 9 die Ausgabe auf Papier.
2. Lass das Programm laufen und vergleiche.
3. Debugger: Setz einen Breakpoint in den Kopierkonstruktor und starte. Welche Zeile in `main` steht im Call-Stack, wenn er zum ersten Mal hält?
4. Erkläre die beiden Überraschungen: Warum gibt es in Zeile 7 kein `C(…)`? Warum steht in Zeile 8 `D(t)` vor `nach`?

A5 – Compiler-Orakel und Kompakt (T3, T7 · Orakel · 20 min)

Teil a – Orakel (`a5_orakel/orakel.cpp`, wie Blatt 2, Aufgabe 2)

Welche der Zeilen A bis Q in `main` übersetzen nicht, und warum? Gehe davon aus, dass jede fehlerhafte Zeile einfach fehlt. Was gibt das Programm dann aus? Achtung: Zeile F verändert `i`.

```
int f(int a, int b = 2) { return a * b; }
int f(double) { return 10; }
int g(int& x) { return ++x; }
int h(const int& x) { return x + 1; }
string s(const string& t, char c = '!') { return t + c; }
void k(long) { cout << "long "; }
void k(double) { cout << "double "; }

int i = 3; const int ci = 4; long l = 5;
f(i)  f(i, 3)  f(2.5)  f(2.5f)  f(l)          // A B C D E
g(i)  g(ci)  g(5)  h(5)  h(ci) und i          // F G H I J
s("Hi")  s("Hi", '?')  s("Hi", "?")           // K L M
k(1L)  k(1)  k(1.0f)  k('a')                  // N O P Q
```

Prüfen: Kommentiere die fehlerhafte Zeile aus, baue neu und lies den nächsten Fehler. Vergleiche die Meldung mit deiner Begründung.

Teil b – Box (`kompakt_box.cpp`, wie Blatt 3, Aufgabe 5/6)

Schreib `Box` so kompakt wie möglich, sodass die Ausgabe `1x1 3x3 3x4 7x7 5x5` entsteht. Gegeben sind `Box a; Box b(3); Box c(3, 4); Box d = 7;` und der Aufruf `zeige(5)`. **Transfer:** Welche Zeilen übersetzen nicht mehr, wenn der Konstruktor mit einem Argument `explicit` wird?

Teil c – Zahl (`kompakt_zahl.cpp`, wie Blatt 3, Aufgabe 4)

Schreib `Zahl` mit `class`, so wenigen Methoden wie möglich und mit Elementinitialisierungsliste. Die Ausgabe soll `8 5 0 2` sein. `z.wert() = 8` muss gehen, `cz.wert() = 1` für ein `const Zahl cz` muss einen Compilerfehler geben.

A6 – Zahlenfolge (T8 · Fehlerjagd · 30 min)

Dateien: `a6_zahlenfolge/Zahlenfolge.h`, `Zahlenfolge.cpp` und drei Testdateien: `test_zf_basis.cpp`, `test_zf_wachsen.cpp`, `test_zf_kopie.cpp`

Eine Folge von `int` in einem selbst verwalteten `int*`-Feld (wie `IntVector` von Blatt 4). `append` hängt an und verdoppelt die Kapazität, wenn das Feld voll ist. Kopierkonstruktor und Zuweisung machen tiefe Kopien, die Zuweisung arbeitet klassisch (freigeben, neu anlegen, kopieren).

Auftrag: 4 Fehler, alle Tests laufen mit `TEST_F(AllTest, …)`.

Vorgehen, das sich hier lohnt: 1. Zuerst nur `test_zf_basis.cpp` im Build lassen (die anderen beiden: *Exclude from Build*). Sind dort alle Tests rot, liegt es an etwas, das jedes Objekt betrifft. Lies die AllTest-Meldungen `freed_not_allocated` und `allocated_not_freed`: Welches Paar aus `new`/`delete` passt nicht? 2. Dann `test_zf_wachsen.cpp`. Spiel `append` für Kapazität 2 mit den Werten 10, 20, 30 auf Papier durch. Im Debugger: Breakpoint auf `delete[] _daten;` in `append`, dann in Expressions `*neu@neue_cap` und `*_daten@_size` vergleichen. 3. Zuletzt `test_zf_kopie.cpp`. Expressions `a._daten` und `b._daten` nach der Kopie vergleichen. Bei der Selbstzuweisung: Worauf zeigt `other._daten`, nachdem `delete[] _daten;` gelaufen ist?

Gegenprobe zum Schluss: valgrind über das Testprogramm, keine Meldung.

Transfer: Schreib den Zuweisungsoperator danach mit Copy & Swap um (Skript 4.4). Warum braucht er dann keinen Identitätstest mehr?

A7 – Thermostat (T4, T9 · Header aus Test · 30 min)

Datei: `a7_thermostat/test_thermostat.cpp`, **nur der Test**.

Ein Thermostat regelt einen Raum. Er hat einen Sollwert (Vorgabe 20 Grad, erlaubt 5 bis 30) und einen Raumnamen (Vorgabe `"Flur"`, nicht leer, nach dem Erzeugen unveränderlich). Ungültige Startwerte werfen einen `std::runtime_error` (Text frei wählbar). `stelle(…)` begrenzt auf 5 bis 30 und meldet, ob der Wert unverändert übernommen wurde. Eine Korrektur gleicht einen ungenauen Fühler aus: Geheizt wird, wenn der Messwert plus Korrektur unter dem Sollwert liegt. Ein Thermostat kann weder kopiert noch zugewiesen werden. Es wird gezählt, wie viele Thermostate aktuell existieren. Eine freie Funktion `synchronisiere` übernimmt den Sollwert eines anderen Thermostats und greift dabei direkt auf die privaten Attribute zu.

Auftrag: 1. **Ziel nach 12 Minuten:** `Thermostat.h` und `Thermostat.cpp` mit Stubs, das Testprojekt **kompiliert und linkt**, auch wenn Tests rot sind. Nutze die Tabelle „Steht im Test → Folgerung“ aus Teil 1.6. Schreib zu jeder Deklaration die Testzeile auf, aus der du sie abgeleitet hast. 2. Danach die Logik füllen, bis alle 7 Tests grün sind.

Transfer: Was passiert mit `anzahl()`, wenn du im Konstruktor zuerst zählst und danach prüfst? Welcher Test merkt es?

A8 – Speicherlayout (T10 · Debugger-Frage · 15 min)

Datei: `a8_speicher/speicher.cpp`, eigenes Konsolenprojekt (wie Blatt 1, Aufgabe 4, und Abb. 2.1 im Skript).

```
struct Messpunkt { char kennung; double x; int nr; short werte[3]; };
Messpunkt feld[5];  const Messpunkt* p1 = &feld[1];  const Messpunkt* p4 = &feld[4];
int (*m)[4] = new int[3][4]{};  int* flach = &m[0][0];  flach[6] = 42;
```

Beantworte auf Papier für 64 Bit (`double` auf 8 Byte ausgerichtet):

Frage	Ausdruck
F1	`sizeof(Messpunkt)`
F2	`(char*)&feld[0].nr - (char*)&feld[0]`
F3	`p4 - p1`
F4	`(char*)p4 - (char*)p1`
F5	`&feld[3].werte[2] - &feld[1].werte[0]`
F6	`(p1 + 2)->nr` und `(p1 + 2)->werte[1]`
F7	`m[1][2]` und `&m[2][1] - &m[0][0]`
F8	`sizeof(m[0])` und `sizeof(m)`

Dann prüfen: Breakpoint auf die Zeile `cout << "HIER"`, *Debug As*, und jeden Ausdruck in die **Expressions**-View eintragen. Erst danach das Programm zu Ende laufen lassen. Zeichne für F2 das Speicherbild eines `Messpunkt` mit allen Füllbytes.

Zusatz: Wie groß ist `Messpunkt` mit `-m32`? (Unter 32 Bit richtet gcc `double` innerhalb von Strukturen nur auf 4 Byte aus.)

A9 – Staffel (T5 · Stufenbau · 40 min)

Dateien: `a9_staffel/test_stufe1_laeufer.cpp`, `test_stufe2_team.cpp`. Die Header schreibst du selbst.

Stufe 1 – `Laeufer` (`Laeufer.h/.cpp`, 10 min): Ein Läufer hat einen Namen (nicht leer) und eine Zeit in Sekunden (Vorgabe 0, nicht negativ). Ungültiges wirft einen `std::runtime_error`. Name und Zeit sind lesend abfragbar, der Name ohne Kopie. `setze_zeit` liefert, ob es geklappt hat, und lässt bei negativer Zeit alles unverändert. `anzeige()` liefert z. B. `"Susi (1:05)"`. → Nur `test_stufe1_laeufer.cpp` im Build lassen. Grün? Dann sichern (Commit oder Ordnerkopie).

Stufe 2 – `Team` (`Team.h/.cpp`, 15 min): Ein Team hat einen Namen und höchstens 4 Läufer, die es als **Kopie** speichert (`std::vector<Laeufer>`). `aufnehmen` liefert `false`, wenn das Team voll ist. `gesamtzeit()` liefert die Summe. `schnellster()` liefert den Läufer mit der kleinsten Zeit ohne Kopie (bei Gleichstand den ersten), bei leerem Team wird `"leer"` geworfen. `laeufer(i)` gibt veränderbaren bzw. für konstante Teams nur lesenden Zugriff, bei ungültigem Index wird `"index out of range"` geworfen. `groesse()` liefert die Anzahl. → Jetzt auch `test_stufe2_team.cpp` dazunehmen. Grün? Sichern.

Stufe 3 – Programm `staffel` (eigenes Projekt, 15 min, wie Blatt 3, Aufgabe 2): Die Läufernamen kommen von der Kommandozeile. Ihre Zeiten erzeugst du mit `create_same_randints(n, 40, 90)`. Die Läufer werden der Reihe nach auf Teams zu je 4 verteilt („Team 1“, „Team 2“, …). Ausgabe: jedes Team mit Gesamtzeit, mit `-v` zusätzlich die Läufer. Danach kommt das schnellste **vollständige** Team mit seinem schnellsten Läufer. Ohne Namen bricht das Programm mit einer Meldung ab.

```
$ ./staffel Rudi Susi Votan Bodo Jonny Hermine Angelika Elli Riki Emanuel
Team 1: 196 s
Team 2: 203 s
Team 3: 127 s
Sieger: Team 1, schnellster Laeufer: Votan (0:44)
```

(Die Zufallswerte gelten für g++ mit der Kurs-`pbma`.)

Transfer: `aufnehmen(const Laeufer&)` speichert eine Kopie. Was müsste sich ändern, damit das Team auf die Original-Läufer zeigt, und welche neue Gefahr entstünde dadurch?

Teil 3 – Rezeptkarten zum Abschreiben und Anpassen

Gerüste für die Teile, die in jeder Aufgabe fast gleich aussehen. Ziel ist nicht Auswendiglernen, sondern sie so oft abzutippen, bis du sie ohne Vorlage kannst.

R1 – Programm mit pbma und Datei (T1)

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
#include "pbma.h"
using namespace std;

vector<int> lies(const string& name) {          // erste Zahl = Anzahl
    ifstream datei(name);
    if (!datei) throw runtime_error("Datei nicht lesbar: " + name);
    size_t anzahl = 0;
    if (!(datei >> anzahl)) throw runtime_error("Anzahl fehlt");
    vector<int> werte;
    int x = 0;
    while (datei >> x) werte.push_back(x);
    if (werte.size() != anzahl) throw runtime_error("Anzahl passt nicht");
    return werte;
}

int main(int argc, const char* argv[]) {
    try {
        args_t args(argc, argv);
        string name = args.pos(0, "daten.dat");     // 1. Positionsargument
        int grenze = args.int_option("grenze", 50); // -grenze=70
        bool verbose = args.flag("v");              // -v  (Schluessel ohne Minus)
        vector<int> w = lies(name);
        if (verbose)
            for (size_t i = 0; i < w.size(); ++i)
                cout << setw(4) << i + 1 << ": " << w[i] << endl;
        long summe = 0;
        int ueber = 0;
        for (int x : w) {
            summe += x;
            if (x >= grenze) ++ueber;
        }
        cout << "Ueber Grenze: " << ueber << endl;
        cout << fixed << setprecision(2)
             << static_cast<double>(summe) / w.size() << endl;
    } catch (const exception& e) {
        cerr << "Fehler: " << e.what() << endl;
        return 1;
    }
}
```

R2 – Header-Gerüst für eine Klasse (T4, T6, T9)

```
#ifndef KLASSE_H
#define KLASSE_H
#include <string>

class Klasse {
public:
    explicit Klasse(int wert = 0, const std::string& name = "x"); // Defaults + explicit
    ~Klasse();
    Klasse(const Klasse&) = delete;              // nur wenn "nicht kopierbar"
    Klasse& operator=(const Klasse&) = delete;

    int wert() const;                            // lesend -> const
    const std::string& name() const;             // ohne Kopie
    int& ref();                                  // veraenderbar
    int ref() const;                             // ... und fuer const-Objekte
    bool setze(int wert);                        // Erfolg melden
    static int anzahl();                         // ohne Instanz

    friend void f(Klasse& a, const Klasse& b);   // direkter Zugriff auf private

private:
    int _wert;
    const std::string _name;                     // unveraenderlich -> Init-Liste
    static int _anzahl;                          // Deklaration
};

void f(Klasse& a, const Klasse& b);
#endif
```

R3 – static-Zähler richtig (T6)

```
int Klasse::_anzahl = 0;                // Definition in der .cpp, ohne static

Klasse::Klasse(int wert, const std::string& name) : _wert(wert), _name(name) {
    if (wert < 0) throw std::runtime_error("invalid value"); // erst pruefen ...
    ++_anzahl;                                               // ... dann zaehlen
}
// Nur falls Kopieren erlaubt ist: Kopien zaehlen mit, sonst stimmt der Zaehler nicht
Klasse::Klasse(const Klasse& o) : _wert(o._wert), _name(o._name) { ++_anzahl; }
Klasse::~Klasse() { --_anzahl; }
int Klasse::anzahl() { return _anzahl; }  // static steht nur im Header
```

R4 – at() als const-Paar mit Prüfung (T2, T8)

```
int& Feld::at(size_t i) {
    if (i >= _size) throw std::runtime_error("index out of range"); // deckt auch -1 ab
    return _daten[i];
}
int Feld::at(size_t i) const {
    if (i >= _size) throw std::runtime_error("index out of range");
    return _daten[i];
}
```

R5 – Dreierregel mit Copy & Swap (T8)

```
// Header:  Feld(const Feld& o);  Feld& operator=(Feld o);  ~Feld();
//          friend void swap(Feld& a, Feld& b) noexcept;
Feld::Feld(size_t n) : _daten(new int[n]{}), _size(n) {}
Feld::Feld(const Feld& o) : _daten(new int[o._size]), _size(o._size) {
    for (size_t i = 0; i < _size; ++i) _daten[i] = o._daten[i];   // tief kopieren
}
void swap(Feld& a, Feld& b) noexcept {
    std::swap(a._daten, b._daten);
    std::swap(a._size, b._size);
}
Feld& Feld::operator=(Feld o) {   // Wertparameter = Kopie
    swap(*this, o);                // tauschen; alter Speicher stirbt mit o
    return *this;                  // Selbst- und Kettenzuweisung gehen automatisch
}
Feld::~Feld() { delete[] _daten; } // new[] -> delete[]
```

R6 – Klassische Zuweisung (wenn so verlangt)

```
Feld& Feld::operator=(const Feld& o) {
    if (this == &o) return *this;          // 1. Identitaetstest
    delete[] _daten;                       // 2. alten Speicher freigeben
    _size = o._size;
    _daten = new int[_size];               // 3. neu anlegen
    for (size_t i = 0; i < _size; ++i) _daten[i] = o._daten[i]; // 4. kopieren
    return *this;                          // 5. fuer a = b = c
}
```

R7 – Löschen im vector während der Schleife

```
size_t i = 0;
while (i < v.size()) {
    if (v[i] == wert) v.erase(begin(v) + i);  // Nachfolger rueckt auf i
    else ++i;                                 // nur weiter, wenn nichts geloescht
}
```

R8 – Ausgabeformate

```
cout << setw(4) << nr;                              // rechtsbuendig, Breite 4
cout << setfill('0') << setw(2) << tag << setfill(' '); // 03, Fuellzeichen zuruecksetzen
cout << fixed << setprecision(2) << 67.2;           // 67.20
double schnitt = static_cast<double>(summe) / n;    // NIE int / int
```

R9 – Eigener Mini-Test ohne Speicherprüfung

```
#include "gtest/gtest.h"
#include "Feld.h"
TEST(Mein, Ausprobieren) {        // TEST statt TEST_F(AllTest, ...)
    Feld f(3);
    f.at(1) = 7;
    std::cout << f.at(1) << std::endl;
    EXPECT_EQ(f.at(1), 7);
}
```

Teil 4 – Fehlerbuch (Vorlage)

Pro gefundenem Fehler eine Zeile. Nach den Aufgaben A1 bis A9 hast du rund 20 Einträge und erkennst deine Muster.

Aufgabe	Symptom (Test, erwartet / tatsächlich)	Ursache	Gefunden mit	Minuten

Nachbesprechung pro Aufgabe: Welcher Fehler hat am längsten gedauert? Welche Papier-Skizze hätte ihn vor dem Bauen verhindert? Welches Werkzeug hat ihn am Ende gezeigt?